UNIVERSIDADE FEDERAL DE SÃO JOÃO DEL-REI

Davi Gomes Pinto Coelho

Metodologia para Decompor uma Aplicação Monolítica em Microsserviços

São João del-Rei

2026

UNIVERSIDADE FEDERAL DE SÃO JOÃO DEL-REI

Davi Gomes Pinto Coelho

Metodologia para Decompor uma Aplicação Monolítica em Microsserviços

Monografia apresentada como requisito da disciplina de Projeto Orientado em Computação I do Curso de Bacharelado em Ciência da Computação da UFSJ.

Orientador: Prof. Dr. Matheus Carvalho Viana

Universidade Federal de São João del-Rei – UFSJ

Bacharelado em Ciência da Computação

São João del-Rei

2026

Davi Gomes Pinto Coelho

Metodologia para Decompor uma Aplicação Monolítica em Microsserviços

Monografia apresentada como requisito da disciplina de Projeto Orientado em Computação I do Curso de Bacharelado em Ciência da Computação da UFSJ.

São João del-Rei, 21 de outubro de 2026:

Prof. Dr. Matheus Carvalho Viana
Orientador

São João del-Rei
2026

Este trabalho é dedicado às crianças adultas que,
quando pequenas, sonharam em se tornar cientistas.

Agradecimentos

Os agradecimentos principais são direcionados à Gerald Weber, Miguel Frasson, Leslie H. Watter, Bruno Parente Lima, Flávio de Vasconcellos Corrêa, Otavio Real Salvador, Renato Machnievscz[1] e todos aqueles que contribuíram para que a produção de trabalhos acadêmicos conforme as normas ABNT com LaTeX fosse possível.

Agradecimentos especiais são direcionados ao Centro de Pesquisa em Arquitetura da Informação[2] da Universidade de Brasília (CPAI), ao grupo de usuários *latex-br*[3] e aos novos voluntários do grupo *abnTeX2*[4] que contribuíram e que ainda contribuirão para a evolução do abnTeX2.

[1] Os nomes dos integrantes do primeiro projeto abnTeX foram extraídos de <http://codigolivre.org.br/projects/abntex/>

[2] <http://www.cpai.unb.br/>

[3] <http://groups.google.com/group/latex-br>

[4] <http://groups.google.com/group/abntex2> e <http://abntex2.googlecode.com/>

"Não vos amoldeis às estruturas deste mundo,
mas transformai-vos pela renovação da mente,
a fim de distinguir qual é a vontade de Deus:
o que é bom, o que Lhe é agradável, o que é perfeito.
(Bíblia Sagrada, Romanos 12, 2)

Resumo

A evolução dos sistemas de informação frequentemente conduz aplicações construídas sob arquiteturas monolíticas a gargalos de acoplamento, complexidade de manutenção e limitações de escalabilidade. A transição para microsserviços surge como alternativa para viabilizar o desacoplamento e a evolução independente dos serviços, mas a escolha da estratégia de decomposição ainda carece de validação empírica consistente na literatura. Este trabalho apresenta um estudo empírico comparativo estruturado em uma matriz 3×3, avaliando o impacto de três metodologias de decomposição arquitetural (por Capacidade de Negócio, por Subdomínio com *Domain-Driven Design* e por Transações com foco em restrições ACID) aplicadas a três sistemas monolíticos de código aberto em TypeScript e NestJS, representativos de domínios distintos: controle de inventário de missão crítica, plataforma de leilões com alta concorrência em tempo real e comércio eletrônico transacional. A migração incremental é conduzida por meio do padrão *Strangler Fig* mediado por proxy reverso NGINX na borda, com coleta de dados quantitativos em quatro marcos de transição. A avaliação contempla quatro categorias de medição: agilidade e DevOps, custo operacional e infraestrutura, desempenho de requisições sob estresse de carga e manutenibilidade estrutural do código.

Palavras-chaves: Microsserviços. Decomposição de Monólitos. Estudo Empírico. Strangler Fig. Desempenho de Software.

Abstract

The evolution of information systems often leads applications built on monolithic architectures to coupling bottlenecks, high maintenance complexity, and scalability limitations. Transitioning to microservices emerges as an alternative to foster decoupling and independent service evolution; however, the choice of decomposition strategy still lacks consistent empirical validation in the literature. This work presents an empirical comparative study structured in a 3×3 matrix, evaluating the impact of three architectural decomposition methodologies (by Business Capability, by Subdomain with Domain-Driven Design, and by Transactions focusing on ACID constraints) applied to three open-source monolithic systems in TypeScript and NestJS, representative of distinct domains: mission-critical inventory management, high-concurrency real-time auctions, and transactional e-commerce. Incremental migration is conducted using the Strangler Fig pattern mediated by an edge NGINX reverse proxy, with quantitative data collection across four transition milestones (snapshots). The evaluation covers four measurement categories: agility and DevOps, operational cost and infrastructure, request performance under stress testing, and structural code maintainability.

Key-words: Microservices. Monolith Decomposition. Empirical Study. Strangler Fig. Software Performance.

Sumário

1 Introdução 10

2 Fundamentação Teórica 11

2.1 Arquitetura Monolítica 11

2.2 Arquitetura de Microsserviços 12

2.3 Padrões de Decomposição 14

2.3.1 Decomposição por Capacidade de Negócio 14

2.3.2 Decomposição por Subdomínio (DDD) 15

2.3.3 Decomposição por Transações 15

2.4 Métricas e Instrumentos de Coleta 16

2.4.1 Métricas de Agilidade e DevOps 16

2.4.2 Métricas de Custo Operacional e Infraestrutura 16

2.4.3 Métricas de Desempenho 16

2.4.4 Métricas de Manutenibilidade e Complexidade 17

2.5 Engenharia de Software Experimental 17

2.6 Ferramentas 18

2.6.1 Docker 18

2.6.2 Madge 18

2.6.3 Autocannon 18

2.6.4 NGINX 18

3 Trabalhos Relacionados 20

4 Metodologia 23

4.1 Desenho da Pesquisa e Abordagem Experimental 23

4.2 Seleção e Justificativa das Aplicações (Estudos de Caso) 24

4.3 Procedimentos de Migração e Marcos de Avaliação 25

5 Desenvolvimento 27

5.1 Arquitetura de Referência 27

5.2 Infraestrutura de Experimentação e Coleta de Métricas 28

5.3 Decomposição por Capacidade de Negócio 30

5.4 Decomposição por Subdomínio (DDD) 32

5.5 Decomposição por Transações 34

5.6 Comparação entre as Metodologias 36

5.7 Síntese do Capítulo 37

6 Conclusão 38
6.1 Contribuições 38
6.2 Limitações 38
6.3 Trabalhos Futuros 38

Referências 39

1 Introdução

2 Fundamentação Teórica

A evolução dos sistemas de informação nas últimas décadas foi acompanhada por transformações significativas nos padrões de arquitetura de software (VELEPUCHA; FLORES, 2023). Com o crescimento da complexidade das aplicações e a exigência de entregas contínuas e escaláveis, os modelos tradicionais de desenvolvimento baseados em aplicações monolíticas passaram a apresentar limitações estruturais significativas. Nesse cenário, o estilo arquitetural de microsserviços consolidou-se como uma abordagem proeminente para a construção de sistemas modernos, modulares e desacoplados (NEWMAN, 2020).

2.1 Arquitetura Monolítica

Um número considerável de sistemas de software foi desenvolvido sob o paradigma monolítico. De acordo com Richardson (2018), uma aplicação monolítica é concebida como uma unidade lógica singular de execução. Em um monólito típico, todos os componentes do sistema, tais como a interface com o usuário, a lógica de negócio e os módulos de acesso a dados, são empacotados e implantados conjuntamente em um único processo computacional.

Embora a arquitetura monolítica apresente vantagens nos estágios iniciais do ciclo de vida do software, como a simplicidade de testes locais, a facilidade de implantação inicial e a alta performance de chamadas de função diretas em memória, ela frequentemente enfrenta severos gargalos à medida que o sistema e a organização expandem-se, conforme cita (NEWMAN, 2021).

Um dos principais problemas decorre do alto acoplamento de código e de banco de dados. A proliferação de dependências diretas entre módulos e o compartilhamento de um único esquema de banco de dados relacional tornam o sistema rígido. Nessa estrutura, pequenas alterações em uma funcionalidade específica podem desencadear efeitos colaterais imprevistos em módulos aparentemente não relacionados (RICHARDSON, 2018).

Adicionalmente, a arquitetura monolítica impõe uma limitação severa à escalabilidade granular. Para escalar uma aplicação monolítica sob alta demanda, é necessário duplicar a unidade inteira de execução em múltiplos servidores. Esse processo consome recursos de infraestrutura de forma ineficiente, visto que módulos ociosos são replicados juntamente com aqueles que realmente necessitam de maior capacidade de processamento (LEWIS; FOWLER, 2014).

Por fim, manifesta-se o acoplamento no ciclo de implantação. Toda a aplicação

precisa ser recompilada, testada integralmente e reimplantada a cada nova atualização, por menor que seja a alteração no código-fonte. Esse fator eleva substancialmente a latência de entrega de novas funcionalidades e aumenta o risco operacional de falha global, onde uma exceção não tratada em um módulo secundário pode derrubar todo o processo da aplicação.

Ressalta-se que, no contexto deste trabalho, um sistema web com arquitetura cliente-servidor (*front-end* e *back-end*), é tratado como sendo monolítico. Apesar de, tecnicamente, um sistema nessa arquitetura gere mais de um processo em execução (normalmente, um processo para o *back-end* e um ou mais processos do *front-end*, cada um em uma máquina de um usuário), cada um desses processos é um sistema monolítico em si, em que todos os requisitos funcionais estão acoplados em uma única unidade de software.

2.2 Arquitetura de Microsserviços

Para suplantar as limitações estruturais do monólito, Lewis e Fowler (2014) definem a arquitetura de microsserviços como uma abordagem em que a aplicação é projetada como um ecossistema de pequenos serviços autônomos. Cada serviço é executado em seu próprio processo computacional e se comunica com os demais por meio de mecanismos leves de rede, tipicamente utilizando APIs fundamentadas no protocolo HTTP com o padrão REST ou barramentos de mensageria assíncrona.

Segundo Newman (2021), microsserviços são unidades de software independentemente implantáveis, fortemente coesas e estruturadas em torno de limites de negócios específicos. Em vez de organizar o código por camadas técnicas horizontais, tais como camada de apresentação, camada de regras de negócio e camada de persistência, os microsserviços alinham a estrutura do software às capacidades de negócio da organização, promovendo o desacoplamento e a clareza de responsabilidade de cada componente. Do ponto de vista da funcionalidade do software, a granularidade de um microsserviço pode variar, podendo contemplar um todo um requisito funcional (cadastro de produto, por exemplo) ou ser equivalente a uma simples função de código (verificação de CPF ou de CEP, por exemplo).

A transição para microsserviços fundamenta-se em um conjunto de características e princípios arquiteturais estruturantes (VELEPUCHA; FLORES, 2023). A primeira dessas características refere-se à componentização promovida por meio de serviços. Conforme enfatizado por Lewis e Fowler (2014), a componentização tradicional via bibliotecas ligadas em memória é substituída por serviços independentes expostos via rede. Essa mudança assegura que a substituição, atualização ou manutenção de um serviço específico ocorra sem a necessidade de recompactar ou reimplantar os demais componentes do ecossistema.

Em adição à substituição de bibliotecas por serviços de rede, sobressai-se a go-

vernança e a descentralização tecnológica. Diferente do monólito, que impõe uma única linguagem e um único *framework* para toda a aplicação, Lewis e Fowler (2014) ressaltam que os microsserviços viabilizam a adoção da abordagem da "ferramenta certa para o trabalho", abrindo espaço para o que Richardson (2018) descreveu historicamente como uma "arquitetura poliglota". Sob essa ótica, Newman (2020) reforça que cada microsserviço é tecnologicamente agnóstico, garantindo autonomia para escolher a linguagem de programação, o *framework* e as bibliotecas que melhor se adaptem às suas exigências técnicas particulares.

No que tange ao gerenciamento da informação, o isolamento de dados por serviço é apontado por Richardson (2018) como um pilar incontornável. Cada microsserviço deve possuir e gerenciar seu próprio repositório de dados privado, sendo impedido o acesso direto de serviços externos ao seu banco de dados. Qualquer interação ou consulta a informações sob domínio de outro serviço deve ocorrer exclusivamente por meio de sua API pública. Esse princípio elimina o acoplamento oculto no nível de esquemas relacionais de banco de dados.

Dessa segregação de dados deriva diretamente a possibilidade de adoção da persistência poliglota. Como o banco de dados deixa de ser um recurso compartilhado globalmente, Richardson (2018) explica que é possível selecionar tecnologias de armazenamento distintas de acordo com a natureza de cada domínio de negócio. Por exemplo, um serviço de catálogo de produtos pode adotar um banco NoSQL orientado a documentos para otimização de leitura, enquanto o serviço de carrinho de compras utiliza um banco chave-valor em memória para alta performance, e o serviço de pedidos utiliza um banco relacional garantidor de transações ACID (RICHARDSON, 2018).

Por fim, a separação física entre processos contribui de maneira determinante para a resiliência operacional e o isolamento de falhas. Em uma arquitetura monolítica tradicional, uma exceção não tratada pode paralisar todo o processo computacional. Em contrapartida, sob o modelo de microsserviços, a degradação ou a indisponibilidade pontual de um serviço específico não provoca o colapso de toda a aplicação. A adoção de padrões de resiliência impede a propagação de falhas em cascata, permitindo que funcionalidades não afetadas continuem operando normalmente para o usuário final (NEWMAN, 2021).

Apesar dos evidentes benefícios de disponibilidade e desacoplamento, a adoção de microsserviços introduz uma nova camada de complexidade inerente aos sistemas distribuídos (LEWIS; FOWLER, 2014; SÖYLEMEZ; TEKINERDOGAN; TARHAN, 2022). Conforme adverte Richardson (2018), a principal contrapartida operacional reside na observabilidade e no diagnóstico de falhas. Enquanto em um monólito a depuração ocorre pela análise de um log único e unificado com a pilha de execução completa (*stack trace*), em microsserviços a execução de uma única requisição de usuário atravessa múltiplos serviços independentes via rede.

Essa pulverização de processos torna a identificação das causas raízes de erros um desafio complexo. Para contornar essa dificuldade e correlacionar os registros de auditoria espalhados entre diversos *containers*, torna-se indispensável a implementação de técnicas de rastreamento distribuído (*distributed tracing*), injetando identificadores únicos de correlação em todas as chamadas de rede (RICHARDSON, 2018). Além disso, destacam-se como desafios a latência de rede adicional, a exigência de roteamento eficiente por meio de *API Gateways* e o gerenciamento de transações entre múltiplos repositórios.

Em particular, a substituição de transações ACID locais por modelos de consistência eventual exige a reformulação do fluxo de dados. A coordenação de estados entre serviços passa a demandar padrões avançados, tais como a Arquitetura Baseada em Eventos (*Event-Driven Architecture*) e o Padrão Saga (RICHARDSON, 2018). Dessa forma, a transição para microsserviços representa uma escolha consciente (*trade-off*): reduz-se a complexidade interna do código-fonte e ganha-se resiliência, em troca do aumento da complexidade de infraestrutura, rastreabilidade e comunicação de rede.

2.3 Padrões de Decomposição

Neste projeto serão utilizados três diferentes padrões de decomposição de softwares monolíticos para microsserviços. Cada um deles é descrito nas seções seguintes.

2.3.1 Decomposição por Capacidade de Negócio

A decomposição por capacidade de negócio (*Decompose by Business Capability*) fundamenta-se na identificação das funções operacionais e processos que a organização executa para atingir seus objetivos (RICHARDSON, 2018). Nesse contexto, uma capacidade de negócio representa o objetivo funcional da organização (o "que"é realizado), em contraste com as tecnologias usadas no momento (o "como"é realizado).

Dessa forma, a conceituação de uma capacidade permanece estável ao longo do tempo, independentemente das transformações nas ferramentas de software ou na infraestrutura. Por exemplo, a capacidade de "Gestão de Estoque"ou de "Processamento de Pedidos"representa uma necessidade operacional contínua de uma organização comercial, quer seja executada por meio de registros manuais, por um sistema monolítico legado ou por microsserviços isolados.

Nessa estratégia, as fronteiras dos microsserviços são alinhadas às áreas funcionais da organização. Em um sistema corporativo, por exemplo, funções como gestão de estoque, processamento de pedidos e faturamento dão origem a serviços independentes. A principal vantagem teórica dessa abordagem consiste no alinhamento direto entre a estrutura do software e a estrutura organizacional, o que favorece a atribuição de equipes autônomas responsáveis pelo ciclo de vida completo de cada serviço (NEWMAN, 2021).

Entretanto, essa abordagem apresenta limitações quando aplicada a sistemas monolíticos legados que utilizam um banco de dados relacional compartilhado. Como diferentes processos de negócio frequentemente acessam e alteram as mesmas tabelas, a divisão dos serviços baseada apenas na capacidade funcional pode exigir a implementação de transações distribuídas e gerar dependências cruzadas entre os novos serviços.

2.3.2 Decomposição por Subdomínio (DDD)

A decomposição por subdomínio baseia-se nos princípios do *Domain-Driven Design* (DDD), propostos por Evans (2003) e aplicados à arquitetura de microsserviços por Richardson (2018). Nessa abordagem, o domínio do sistema é particionado em subdomínios, classificados em centrais, de suporte e genéricos, sendo cada um associado a um Contexto Delimitado (*Bounded Context*).

O conceito de *Bounded Context* estabelece que cada modelo de dados e vocabulário técnico possui significado estrito dentro de seus limites lógicos (EVANS, 2003). Por exemplo, a entidade "Cliente"no contexto de vendas possui atributos e regras de negócio distintos da mesma entidade no contexto de atendimento ou faturamento.

No contexto experimental deste trabalho, a decomposição por subdomínio orienta a separação física dos serviços com base no isolamento dos modelos de dados. Essa estratégia busca reduzir o acoplamento conceitual e assegurar que cada microsserviço mantenha a consistência de seu modelo dentro de um repositório de dados privado, eliminando o acesso direto a bancos de dados compartilhados (NEWMAN, 2020).

2.3.3 Decomposição por Transações

A decomposição por transações (*Decompose by Transactions*) adota um critério técnico focado na consistência dos dados para a definição das fronteiras dos microsserviços (RICHARDSON, 2018). Diferente das abordagens orientadas a processos ou domínios de negócio, essa estratégia prioriza a análise das restrições ACID (*Atomicity, Consistency, Isolation, Durability*) e do fluxo de operações de escrita no banco de dados.

Na refatoração de uma aplicação monolítica, a divisão de um banco de dados relacional centralizado pode transformar transações locais em transações distribuídas entre diferentes serviços. A coordenação dessas transações via rede adiciona complexidade e reduz a vazão do sistema, exigindo o uso de padrões como Saga ou commit de duas fases (*Two-Phase Commit - 2PC*) (RICHARDSON, 2018; NEWMAN, 2021).

Para evitar a degradação de desempenho e a complexidade operacional, a decomposição por transações agrupa no mesmo microsserviço os módulos e tabelas que exigem escrita atômica e consistência imediata. Operações que suportam consistência eventual são isoladas em serviços assíncronos. Na matriz experimental desta pesquisa, essa abordagem

atua como contraponto técnico voltado à redução de chamadas de rede e à preservação da integridade transacional.

2.4 Métricas e Instrumentos de Coleta

Neste projeto, para avaliar os três padrões de decomposição citados, serão adotadas um conjunto de métricas, cuja descrição de cada uma é apresentada nesta seção.

2.4.1 Métricas de Agilidade e DevOps

Para avaliar o impacto da decomposição na operabilidade e no ciclo de vida do software, esta dimensão foca na agilidade de entrega e na inicialização dos serviços:

- **Tempo de Construção:** Mensura a duração total dos processos de empacotamento e geração das imagens Docker para cada microsserviço ou snapshot, avaliando a granularidade do pipeline de CI/CD.
- **Tempo de Inicialização:** Registra o intervalo necessário desde o comando de execução até a prontidão completa do serviço para receber requisições de rede (prontidão de escuta nas portas configuradas).

2.4.2 Métricas de Custo Operacional e Infraestrutura

O consumo de recursos computacionais é monitorado de forma isolada por contêiner para mensurar o *overhead* introduzido pela distribuição da arquitetura:

- **Uso de CPU e Memória:** Coletados por meio de ferramentas de monitoramento de contêineres, registrando a média e o pico de consumo de processamento e RAM durante a execução sob carga.
- **Uso de Armazenamento:** Avalia o impacto do particionamento e da duplicação de dependências analisando o tamanho final das imagens Docker geradas para cada componente da arquitetura.

2.4.3 Métricas de Desempenho

Para avaliar o comportamento dinâmico e o impacto de gargalos operacionais gerados pela transição arquitetural, a pesquisa emprega métricas de desempenho, com ênfase na latência de API e na vazão máxima (*throughput*). O fundamento empírico dessas medições baseia-se na identificação de restrições de desempenho provocadas pela conversão de dependências locais em chamadas de rede. Conforme apontam Richardson (2018) e

Newman (2020), a introdução da comunicação interprocessos (IPC) nas fronteiras dos microsserviços adiciona sobrecarga temporal que afeta diretamente a eficiência do sistema.

A latência é quantificada por meio de percentis, que revelam o comportamento sob estresse e a estabilidade nos tempos de resposta da aplicação. Ademais, a vazão máxima mensura o volume limite de requisições processadas por segundo sob condições controladas de concorrência. Nesse sentido, essas métricas permitem correlacionar diretamente a eficiência operacional da aplicação com o grau de acoplamento distribuído e a sobrecarga de rede imposta por cada abordagem de decomposição.

2.4.4 Métricas de Manutenibilidade e Complexidade

Para avaliar a evolução estrutural da arquitetura ao longo da refatoração, a pesquisa emprega métricas voltadas à manutenibilidade e complexidade, com destaque para as Linhas de Código (LOC), a contagem de componentes físicos e o cruzamento de fronteiras. O fundamento teórico para a análise desse acoplamento baseia-se no conceito de conascência (*connascence*), definido formalmente como a interdependência na qual a alteração em um elemento de software exige a modificação de outro para preservar a corretude do sistema (PAGE-JONES, 1992). O objetivo central do *design* de arquiteturas modulares é a minimização da conascência e do cruzamento de fronteiras entre domínios distintos.

No monólito, o cruzamento de fronteiras é observado como conascências estáticas de nome e tipo, quantificadas por meio de análise estática do código-fonte. Durante a transição para microsserviços por meio do padrão *Strangler Fig*, a remoção dessas dependências físicas evita o acoplamento indesejado que, conforme alertam Newman (2020) e Richardson (2018), converte simples chamadas locais de código em comunicações síncronas via rede (*Inter-Process Communication* - IPC). Além disso, o rastreamento das Linhas de Código (LOC) e do número de componentes físicos permite mensurar o esforço de refatoração, o dimensionamento dos artefatos e o impacto do isolamento em cada estágio da pesquisa.

2.5 Engenharia de Software Experimental

A condução de pesquisas empíricas na área de computação exige o rigor metodológico característico da Engenharia de Software Experimental (WOHLIN et al., 2012). Conforme discutem Kitchenham, Pfleeger e Pickard (2002), abordagens experimentais permitem avaliar tecnologias, métodos e ferramentas de software em ambientes controlados, fornecendo evidências quantitativas sobre os seus reais impactos, benefícios e limitações.

No contexto de refatoração arquitetural, a experimentação controlada por meio de matrizes comparativas e métricas reprodutíveis, como análise estática de código e

testes de carga dinâmicos, é fundamental para mitigar vieses subjetivos. Isso possibilita que decisões de migração de monólitos para microsserviços sejam sustentadas por dados empíricos sólidos e mensuráveis.

2.6 Ferramentas

2.6.1 Docker

A plataforma de conteinerização *Docker* foi adotada para garantir o isolamento consistente dos ambientes de execução em todas as fases da pesquisa. Além de prover portabilidade e padronizar o empacotamento dos serviços, o ambiente conteinerizado viabiliza a coleta controlada de métricas de infraestrutura, permitindo extrair o consumo computacional de CPU e o uso de memória RAM por meio do comando *docker stats* durante os testes experimentais.

2.6.2 Madge

O *Madge* é uma ferramenta de análise estática de código-fonte utilizada para rastrear, analisar e visualizar grafos de dependência em projetos desenvolvidos em JavaScript e TypeScript. No escopo deste trabalho, o *Madge* é empregado para quantificar o acoplamento estático e o cruzamento de fronteiras (*cross-domain calls*), permitindo auditar o número de importações interdomínio que configuram a conascência de código no monólito base e nas etapas de refatoração.

2.6.3 Autocannon

O *Autocannon* é uma ferramenta de teste de carga e estresse de alta performance voltada para aplicações HTTP em Node.js. Na matriz experimental desta pesquisa, o *Autocannon* atua como o motor de estresse dinâmico para avaliar o desempenho das APIs em cada *snapshot* arquitetural, permitindo mensurar métricas críticas como a latência de resposta e a vazão máxima de requisições por segundo sob condições controladas de concorrência.

2.6.4 NGINX

O *NGINX* é um servidor web e proxy reverso de alta performance, projetado para lidar com alto volume de conexões simultâneas com baixo consumo de memória. No escopo desta pesquisa, o *NGINX* atua como o mecanismo operacional do padrão *Strangler Fig*, posicionado na borda da arquitetura para interceptar e redistribuir o tráfego de rede entre a aplicação monolítica e os microsserviços extraídos. Por meio de regras de roteamento

baseadas em prefixos de rotas (URIs), o proxy permite direcionar chamadas para os novos contêineres sem exigir modificações nos clientes consumidores.

3 Trabalhos Relacionados

Nesta seção, são analisados estudos anteriores que investigaram abordagens de decomposição de aplicações monolíticas, técnicas de identificação de microsserviços e avaliações comparativas de estratégias de migração arquitetural. Cada trabalho é apresentado quanto ao seu objetivo, à técnica empregada e aos resultados alcançados, seguido de uma discussão sobre como este trabalho se posiciona em relação às lacunas identificadas.

Levcovitz, Terra e Valente (2016) propõem uma técnica para extração de microsserviços a partir de sistemas corporativos monolíticos de grande porte, validada sobre um sistema bancário legado de aproximadamente 750 mil linhas de código. A técnica analisa o esquema relacional do banco de dados, identificando agrupamentos de tabelas fortemente conectadas por chaves estrangeiras e por transações que operam em conjunto, sem exigir a reescrita do código-fonte da aplicação original. Como resultado, os autores demonstram que é possível identificar automaticamente candidatos a microsserviços coerentes com domínios de negócio reais a partir apenas da estrutura de dados, validando a viabilidade de uma decomposição orientada a transações mesmo em sistemas legados de grande escala.

Kazanavičius e Mažeika (2020) comparam a decomposição de uma mesma aplicação corporativa legada utilizando diferentes métodos de identificação de microsserviços. A metodologia consiste em aplicar múltiplas heurísticas de particionamento ao mesmo sistema e definir critérios de avaliação para julgar as vantagens e desvantagens de cada resultado obtido. Os autores concluem que metodologias distintas produzem particionamentos significativamente diferentes do mesmo sistema, cada um com trade-offs próprios, e propõem um conjunto de critérios qualitativos para orientar a escolha da estratégia mais adequada a um determinado contexto.

Wang, Bornais e Rubin (2024) comparam, de forma independente, oito ferramentas automáticas de decomposição de microsserviços. Os autores aplicam essas ferramentas a quatro aplicações monolíticas selecionadas sistematicamente: três delas já haviam sido utilizadas em pesquisas anteriores sobre decomposição, e uma quarta aplicação foi incluída propositalmente por nunca ter sido decomposta antes, para testar a generalização das ferramentas. Combinando métricas quantitativas de modularidade e de similaridade estrutural com entrevistas junto a desenvolvedores responsáveis por duas das aplicações, os autores constatam que nenhuma ferramenta se destaca de forma consistente em todos os cenários avaliados, isto é, o desempenho de cada uma depende fortemente de características específicas da aplicação analisada. Além disso, destacam que nenhuma das ferramentas avaliadas produz uma decomposição efetivamente executável.

Barzotto e Farias (2022) investigam, por meio de um estudo de caso empírico na in-

dústria financeira, os impactos da decomposição de um módulo crítico (saque em dinheiro em terminais de autoatendimento) de uma aplicação monolítica em Java/Spring para uma arquitetura baseada em microsserviços. Os autores compararam a versão monolítica com a versão decomposta utilizando métricas clássicas de modularidade de código orientado a objetos e testes de estresse de carga via Gatling para medir o consumo de hardware. Os resultados apontaram ganhos de modularidade e redução no consumo computacional após a decomposição. Entretanto, os autores reportaram limitações técnicas na medição de hardware do sistema monolítico legado e avaliaram apenas uma única aplicação com um método de decomposição pontual direcionado por DDD.

Tabela 1 – Comparação dos trabalhos relacionados

Trabalho	**Objetivo**	**Técnica**	**Resultado principal**	**Lacuna em relação a este trabalho**
Levcovitz, Terra e Valente (2016)	Extrair microsserviços de um monólito	Análise do esquema relacional	Agrupou tabelas coerentes em 1 sistema bancário	1 sistema; sem migração real; sem métricas dinâmicas
Kazanavičius e Mažeika (2020)	Comparar metodologias no mesmo sistema	Heurísticas de particionamento + critérios qualitativos	Metodologias distintas geram partições distintas	1 sistema; avaliação qualitativa
Wang, Bornais e Rubin (2024)	Comparar ferramentas automáticas	8 ferramentas em 4 benchmarks + entrevistas	Nenhuma ferramenta domina; depende da aplicação	Ferramentas automáticas, não metodologias manuais
Abgaz et al. (2023)	Mapear o estado da arte	Revisão sistemática (35 estudos)	Poucos estudos comparam metodologias no mesmo sistema	Revisão; sem dados empíricos próprios
Barzotto e Farias (2022)	Avaliar impactos de decomposição na indústria	Estudo de caso (Java/Spring) + métricas CK + Gatling	Microsserviços apresentaram maior modularidade e menor consumo de recursos	1 sistema; 1 metodologia; medições incompletas no monólito
Este trabalho	Comparar 3 metodologias em 3 sistemas	Estudo empírico comparativo (3×3) + Strangler Fig + métricas estáticas e dinâmicas	*(Cap. 5)*	—

Fonte: Elaborado pelo autor.

A Tabela 1 destaca as principais diferenças entre os trabalhos relacionados e esta pesquisa. Os estudos relacionados revelam lacunas centrais que este trabalho busca explo-

rar. Primeiramente, conforme apontado por Abgaz et al. (2023), a literatura concentra-se predominantemente em sistemas sintéticos ou de benchmark; Wang, Bornais e Rubin (2024) reforçam essa preocupação ao mostrar que o desempenho das técnicas de decomposição depende fortemente de características específicas de cada aplicação, o que limita a generalização de conclusões obtidas sobre um conjunto restrito de benchmarks tradicionais. Este trabalho utiliza três aplicações de código aberto desenvolvidas de forma independente, com arquitetura e complexidade representativas de sistemas reais em produção. Além disso, enquanto Kazanavičius e Mažeika (2020) e Barzotto e Farias (2022) limitam suas investigações empíricas a uma única aplicação e a abordagens isoladas de decomposição, este trabalho estabelece um estudo empírico comparativo em um desenho fatorial 3×3, avaliando sistematicamente três metodologias de decomposição sobre três sistemas de domínios distintos. Por fim, ao contrário de abordagens puramente analíticas como a de Levcovitz, Terra e Valente (2016), este trabalho efetivamente executa e instrumenta as migrações incrementais mediadas pelo padrão *Strangler Fig*, combinando métricas estáticas de modularidade com ensaios dinâmicos de desempenho e observabilidade de infraestrutura conteinerizada ao longo de múltiplos marcos de transição (*snapshots*).

4 Metodologia

Este capítulo descreve a metodologia de pesquisa adotada para conduzir este estudo empírico comparativo sobre estratégias de decomposição de monólitos em microsserviços. A pesquisa fundamenta-se nas diretrizes da Engenharia de Software Experimental, estruturando-se a partir da refatoração e instrumentação sistemática de três aplicações monolíticas de código aberto, submetidas a três metodologias distintas de decomposição arquitetural em uma matriz 3×3.

A estrutura metodológica foi desenhada para coletar e analisar dados empíricos quantitativos ao longo do processo de migração, utilizando o padrão *Strangler Fig* como mecanismo unificado de extração incremental. A observação empírica ocorre de forma iterativa por meio de marcos formais de transição (*snapshots*), permitindo avaliar o impacto de cada estratégia sob quatro dimensões objetivas: agilidade/DevOps, custo operacional e infraestrutura, desempenho sob carga e manutenibilidade estrutural.

As seções a seguir detalham o desenho experimental do estudo empírico comparativo, a caracterização e seleção dos estudos de caso, os critérios de decomposição, os procedimentos operacionais de transição por *snapshots* e os instrumentos de coleta de dados.

4.1 Desenho da Pesquisa e Abordagem Experimental

A pesquisa é orientada pelo paradigma da Engenharia de Software Experimental, adotando uma estrutura 3×3 que cruza três objetos de estudo distintos com três tratamentos arquiteturais independentes. Essa divisão permite avaliar se os efeitos decorrentes de cada estratégia de decomposição são consistentes entre domínios heterogêneos de aplicação ou se dependem fortemente das características intrínsecas do sistema sob refatoração.

O experimento fundamenta-se na definição explícita de variáveis independentes e dependentes:

- **Variáveis Independentes:**

  1. Metodologia de Decomposição Arquitetural: Decomposição por Capacidade de Negócio, Decomposição por Subdomínio (*Domain-Driven Design*) e Decomposição por Transações (restrições ACID).
  2. Domínio e Características da Aplicação: Sistema de Inventário, Sistema de Leilões e Sistema de E-commerce.

- **Variáveis Dependentes:** O impacto de cada abordagem é quantificado por meio das quatro categorias de medição definidas na Seção 2.4:

  1. Agilidade e DevOps: Tempo de compilação/construção de imagens Docker e tempo de inicialização dos serviços até a prontidão de rede;
  2. Custo e Infraestrutura: Consumo de processamento (CPU), consumo de memória RAM e volume de armazenamento ocupado por imagens conteinerizadas;
  3. Desempenho Sob Carga: Vazão máxima de requisições por segundo e percentis de latência de resposta (p_{50}, p_{90}, p_{99}) sob concorrência controlada;
  4. Manutenibilidade e Acoplamento: Linhas de Código (LOC), contagem de componentes físicos e violações de fronteiras lógicas.

O protocolo experimental prevê a execução de nove trilhas de refatoração independentes, partindo do mesmo ponto de partida para cada aplicação. Cada trilha é submetida a testes padronizados de estresse sintético e avaliação estática de código a cada tranformação.

4.2 Seleção e Justificativa das Aplicações (Estudos de Caso)

Para assegurar a validade externa e a relevância prática dos resultados empíricos, evitou-se o uso de sistemas de teste puramente artificiais. Com base nisso, foram selecionadas três aplicações monolíticas de código aberto desenvolvidas em TypeScript com o ecossistema NestJS, representativas de classes arquiteturais e requisitos não funcionais distintos encontrados na indústria de software.

- **Aplicação Crítica: Sistema de Inventário.** A aplicação *inventory-api*[1] representa um sistema de controle de inventário e execução operacional, análogo aos sistemas MES (*Manufacturing Execution System*) industriais. Nesse contexto, é uma aplicação que demanda acesso em tempo real, de modo a não tolerar tempos elevados fora do ar. Além disso, a integridade transacional é estrita: transações operacionais de baixa e transferência de materiais exigem consistência imediata entre armazéns, produtos e saldos em estoque, sendo inadmissível o registro de quantidades negativas.

- **Aplicação de Alta Concorrência: Monitor de Mercado e Leilões.** A aplicação *auction*[2] reflete o comportamento de plataformas de alta performance voltadas a leilões de cartas colecionáveis e negociação de ativos em tempo real. O domínio impõe elevada concorrência de requisições síncronas de lances (*bids*), atualização de

[1] Disponível em: <https://github.com/egp1020/inventory-api>.
[2] Disponível em: <https://github.com/schoibs/auction>.

preços em milissegundos e dispersão de eventos para múltiplos clientes conectados via WebSockets.

- **Aplicação Transacional e Catálogo: E-commerce.** A aplicação *ecommerce*[3] representa o arquétipo do varejo digital clássico, caracterizado por assimetria entre fluxos de consulta e fluxos de escrita. Por um lado, o catálogo de produtos exige alta vazão de leitura com dados desnormalizados; por outro, as rotas de carrinho e liquidação de pedidos demandam coordenação de reservas, cálculo de taxas e integração com serviços auxiliares.

4.3 Procedimentos de Migração e Marcos de Avaliação

A condução prática das refatorações segue uma abordagem evolutiva controlada, mitigando interrupções no funcionamento global do sistema e permitindo aferir os impactos parciais de cada etapa de extração.

A transição arquitetural de cada monólito para microsserviços é mediada pelo padrão *Strangler Fig* (FOWLER, 2004), suportado por um proxy reverso configurado na borda com o software NGINX.

O proxy atua como um ponto único de entrada de tráfego de rede para a aplicação. Nos estágios iniciais, a totalidade das requisições externas é encaminhada diretamente ao monólito. Conforme módulos específicos são extraídos para microsserviços autônomos conteinerizados, as regras de roteamento baseadas em prefixos de caminhos de URI (*path-based routing*) são reconfiguradas dinamicamente no proxy. O tráfego direcionado aos módulos refatorados passa a ser interceptado e despachado para os novos contêineres, sem que o monólito precise ser completamente desativado ou reescrito de forma abrupta.

Para assegurar a rastreabilidade experimental e a auditabilidade dos dados gerados, cada uma das 9 trilhas é particionada em quatro estados temporais, denominados *snapshots*:

1. **Snapshot 0 (Baseline Monolítico):** Corresponde ao código original da aplicação monolítica em execução, sem qualquer intervenção arquitetural na base de serviços. Serve como linha de base referencial para todas as métricas quantitativas;

2. **Snapshot 1 (Primeira Extração Incremental):** Representa a extração inicial de um módulo funcional selecionado segundo as diretrizes de cada metodologia (tipicamente um serviço de menor acoplamento ou de suporte), operando de forma desacoplada atrás do proxy reverso;

[3] Disponível em: <https://github.com/sappChak/nestjs-ecommerce>.

3. **Snapshot 2 (Segunda Extração Incremental):** Representa a evolução do estrangulamento com a segregação de um segundo domínio ou capacidade operacional, ampliando as fronteiras de rede e a comunicação interserviços;

4. **Snapshot Final (Decomposição Concluída):** Estado final em que todos os módulos previstos pela respectiva metodologia foram completamente desacoplados em microsserviços autônomos com persistência isolada, desativando os componentes migrados do monólito legado.

Cada marco de transição é fixado por meio de tags no repositório de versionamento de código e submetido aos mesmos cenários de estresse de carga e amostragem de telemetria, para que os dados de desempenho, custo e acoplamento sejam correlacionados com a evolução da arquitetura.

5 Desenvolvimento

5.1 Arquitetura de Referência

Nessa seção, é apresentada a arquitetura base das aplicações utilizadas na pesquisa. Com base nisso, avaliam-se os módulos, fronteiras e interações entre as partes de cada sistema, de modo que seja possível identificar os pontos de emprego de cada uma das metodologias sem ferir o funcionamento e o fluxo das aplicações.

A padronização na linguagem TypeScript e na plataforma Node.js/NestJS assegura consistência nas comparações de Linhas de Código (LOC), no empacotamento em contêineres e permite uma comparação justa entre os tempos de inicialização e os perfis de consumo de recursos.

Sistema de Inventário

O sistema de inventário foi desenvolvido como um monólito modular em NestJS, seguindo os princípios de Arquitetura Hexagonal e *Domain-Driven Design*. A aplicação é organizada em quatro camadas por contexto: interface, aplicação, domínio e infraestrutura. Os seis contextos delimitados são autenticação, usuários, armazéns, produtos, movimentações e relatórios. Cada contexto possui seus próprios controladores REST, casos de uso, entidades e adaptadores de persistência.

O núcleo compartilhado concentra as preocupações da base da aplicação, como autenticação e autorização, tratamento de exceções e acesso a dados. A persistência apoia-se em um único banco relacional, acessado por um mapeador objeto-relacional, e as operações que alteram o estoque exigem garantias transacionais.

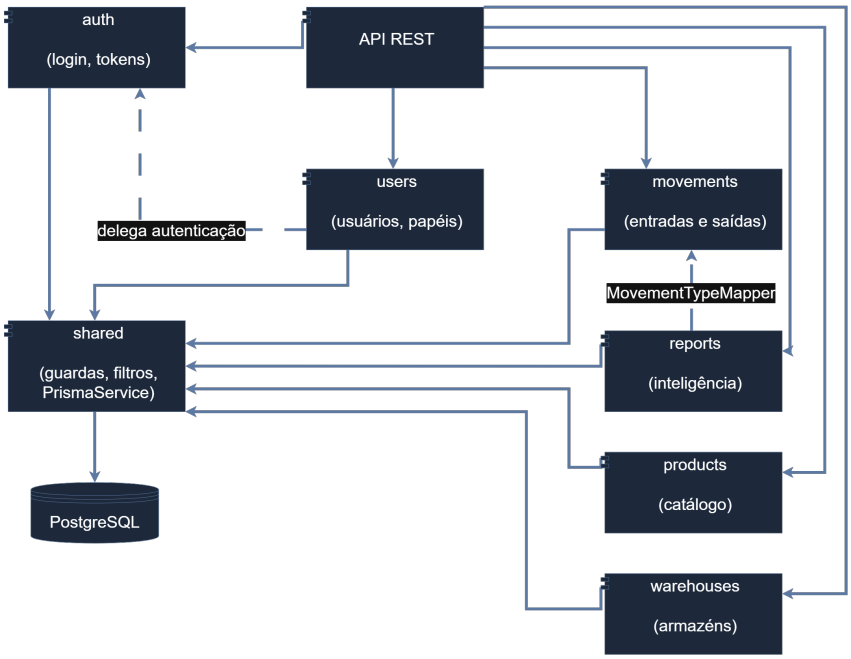
A Figura 1 demonstra a arquitetura base do sistema.

A Tabela 2 reúne as métricas do sistema de inventário no estado inicial (Snapshot 0 canônico), adotado como referência para o cálculo das variações das três metodologias de decomposição.

Sistema de Leilões

O monólito utiliza o ORM TypeORM conectado ao PostgreSQL, auxiliado pelo Redis para controle de filas e processamento de tarefas em segundo plano via BullMQ e transmissão de mensagens em tempo real com Socket.IO. A coexistência de componentes com ciclo de vida assíncrono e de operações transacionais críticas de encerramento de lance

Figura 1 – Arquitetura base do sistema de inventário.



Elaborado pelo autor (2026).

e transferência de ativos permite observar a latência sob estresse e a vazão de requisições durante a partição dos serviços.

E-commerce

Desenvolvida sobre o NestJS com o ODM Mongoose conectado a um banco de dados orientado a documentos (MongoDB), a base de código organiza-se em mais de 20 controladores modulares que cobrem autenticação, produtos, carrinho, pedidos, estoque e avaliações. Sua estrutura permite investigar os impactos da decomposição em modelos orientados a documentos e na gestão de consistência eventual das operações de finalização de compras.

5.2 Infraestrutura de Experimentação e Coleta de Métricas

Para comparar as metodologias de forma precisa, é necessário configurar o ambiente para que os testes sejam feitos em cima das mesmas condições. Nesse sentido, antes de comparar as métricas, foi construída uma infraestrutura de automação, responsável por subir as aplicações, gerar a carga e coletar as métricas. Essa infraestrutura concentra os procedimentos de execução e de coleta em um único conjunto de scripts, de modo que para realizar um novo teste, não é necessário refazer manualmente o processo.

Com base nisso, as medições foram realizadas em um ambiente padrão: Windows 11, processador AMD Ryzen 7 5700X3D, 32 GB de memória, Docker Engine 29.7.2 e Doc-

Tabela 2 – Métricas do sistema de inventário no baseline canônico (Snapshot 0).

Categoria	**Métrica**	**Valor**
Agilidade e DevOps		
	Tempo de construção (média)	46.746 ms
	Tempo de inicialização (média)	9.155 ms
Custo e Infraestrutura		
	CPU total (repouso / carga)	2,22 % / 241,0 %
	RAM total (repouso / carga)	86,3 MB / 188,5 MB
	Imagem da aplicação	151,6 MB
Desempenho		
	`products` — vazão	557,2 req/s
	`products` — latência p50/p90/p99	34,7 / 38,7 / 45,0 ms
	`reports/movements` — vazão	350,8 req/s
	`reports/movements` — latência p50/p90/p99	55,0 / 61,0 / 105,3 ms
	`reports/stock` — vazão	201,3 req/s
	`reports/stock` — latência p50/p90/p99	97,7 / 103,3 / 114,0 ms
	`movements` — vazão	311,9 req/s
	`movements` — latência p50/p90/p99	63,0 / 72,3 / 89,0 ms
Manutenibilidade		
	Linhas de Código	8.783
	Componentes / contêineres	2 / 2
	Chamadas entre fronteiras	69 de 389

Fonte: medição automatizada do experimento (média de três execuções).

ker Compose v5.4.0. Manter o ambiente permite comparar tempos de construção e perfis de consumo de recursos entre snapshots. Também, foram adotadas regras de isolamento, como manter os recursos atribuídos ao Docker constantes e encerrar contêineres externos ao experimento antes de cada medição, para não afetar a amostragem de processamento e de memória nem a contagem de componentes.

A execução de cada aplicação se apoia em contêineres Docker orquestrados por `docker compose`. Cada aplicação sobe com seus serviços de apoio, como banco de dados e, quando aplicável, cache e proxy reverso. Para que a referência base das aplicações iniciais e decomposições partam do mesmo estado, um script de *seed* determinístico popula o banco com um conjunto equivalente de dados de demonstração. As portas são fixadas por aplicação, conforme uma tabela de portas própria para o experimento, de modo a evitar colisões e preservar a reprodutibilidade.

A coleta cobre oito métricas, distribuídas nas quatro categorias de medição definidas no Capítulo 4: tempo de construção da imagem, tempo de inicialização, consumo de processamento e de memória em repouso e sob carga, tamanho das imagens, vazão máxima e percentis de latência sob carga, Linhas de Código, contagem de componentes físicos e violações de fronteiras lógicas. A carga sintética é gerada com a ferramenta *autocannon*. A contagem de Linhas de Código e as violações de fronteiras lógicas vêm de análise estática.

Cada métrica é amostrada três vezes, e o conjunto de resultados brutos e processados é armazenado por aplicação, metodologia e snapshot. O Snapshot 0 canônico é medido uma única vez e reutilizado como referência comum no cálculo das variações; eventuais remedições de baseline ficam registradas nos dados brutos, mas não alteram a referência. As tabelas e figuras apresentadas neste capítulo são geradas automaticamente a partir desses arquivos, o que mantém a rastreabilidade entre a medição e os valores apresentados.

5.3 Decomposição por Capacidade de Negócio

Sistema de Inventário

Sob a decomposição por capacidade de negócio, os seis módulos foram agrupados pelo que a organização faz de fato. Nesse sentido, o agrupamento resultou em três capacidades, sendo elas a capacidade de Administração/Identidade, Controle de Estoque e Relatórios. A capacidade de Relatórios foi a primeira a ser extraída, por concentrar apenas consultas de leitura e não possuir dependências de entrada, ou seja, nenhum outro módulo a importa. No monólito referência, a análise estática registrou 69 dependências cruzando fronteiras de módulo, de um total de 389, sendo a maior parte delas apontando para o núcleo compartilhado.

A extração seguiu o padrão *Strangler Fig* mediado por NGINX, com roteamento por prefixo de caminho: as requisições para `/api/reports/` passaram a ser encaminhadas ao novo serviço e as demais permaneceram no monólito. Adotou-se o banco de leitura compartilhado, com o serviço apontando para o mesmo PostgreSQL apenas para leitura. Como dificuldade, o módulo de relatórios dependia do mapeamento de tipos de movimentação (`MovementTypeMapper`), isto é, o único cruzamento entre domínios no monólito, e foi resolvido por uma cópia local do mapeamento, funcionando como camada anticorrupção.

A segunda capacidade extraída foi a de Administração/Identidade, que reúne a autenticação e o cadastro de usuários. Essa capacidade é pré-requisito de todos os módulos, pois cada controlador depende dos guardas de autenticação. Para extraí-la sem interromper o restante, o serviço de Identidade passou a responder por `/api/auth` e `/api/users`, enquanto o monólito manteve apenas uma cópia local dos guardas de token. Como a validação do token é feita de forma isolada, o monólito não precisa consultar a Identidade a cada requisição. Nessa fase, a persistência ainda era compartilhada entre os serviços.

A terceira e última extração foi a da capacidade de Controle de Estoque, que agrupa armazéns, produtos e movimentações, e o monólito foi desativado. Nessa etapa adotou-se a persistência isolada prevista na metodologia: a Identidade e o Estoque passaram a ter bancos próprios, e o serviço de Relatórios deixou de ter banco. Como os

serviços já não compartilham tabelas, os acessos cruzados passaram a ocorrer por chamadas HTTP. A Identidade valida o armazém no Estoque ao cadastrar um usuário, e o Estoque valida o usuário na Identidade ao registrar uma movimentação, guardando o e-mail do usuário junto ao movimento. Essa separação evidenciou acoplamentos de dados que o banco compartilhado mantinha ocultos e foi a principal dificuldade da trilha. O serviço de Relatórios, por sua vez, consulta os dados do Estoque e os agrega em memória.

A Tabela 3 apresenta as métricas ao longo das quatro fases e a Tabela 4 consolida a variação acumulada da metodologia. O tempo de construção subiu de cerca de 47 segundos no baseline canônico para 138 segundos no estado final, como consequência da construção de imagens independentes. O consumo de memória e de processamento cresceu de forma contínua, com aumento significativo do processamento sob carga na fase final, quando passaram a existir dois bancos dedicados. A vazão média das rotas caiu após a primeira extração, em razão do salto adicional pelo proxy, e se recuperou nas fases seguintes. O número de componentes subiu de dois para seis. As Linhas de Código cresceram até a segunda extração, por duplicação de código transversal, e caíram no estado final, quando o monólito deixou de compor a solução. As chamadas entre fronteiras medidas pela análise estática variaram pouco, pois o acoplamento entre serviços passou a ocorrer pela rede e não é capturado por essa análise.

Tabela 3 – Inventário — métricas por snapshot (Capacidade de Negócio).

Métrica	**S0**	**S1**	**S2**	**Final**
Agilidade				
Tempo de construção (ms)	46.746	116.353	108.405	138.371
Tempo de inicialização (ms)	9.155	10.594	13.166	12.051
Custo				
CPU total (repouso / carga, %)	2,22 / 241	2,78 / 242,65	6,24 / 255,44	6,6 / 405,78
RAM total (repouso / carga, MB)	86,28 / 188,47	149,36 / 306,32	186,85 / 352,72	203,6 / 445,1
Imagens da aplicação (MB)	151,6	363,3	551,1	586,2
Componentes / contêineres	2 / 2	4 / 4	5 / 5	6 / 6
Desempenho				
Vazão (média das rotas, req/s)	355,3	227,4	266,3	360,1
Latência p50 (média das rotas, ms)	62,6	92,7	77,9	55,2
Manutenibilidade				
Linhas de Código	8.783	9.165	9.796	9.599
Chamadas entre fronteiras	69 de 389	73 de 402	78 de 414	78 de 416

Fonte: medição automatizada do experimento (média de três execuções). S0 = baseline canônico (Capacidade de Negócio / S0); demais snapshots medidos nesta trilha.

Tabela 4 – Variação acumulada por aplicação — Capacidade de Negócio (S0 canônico → último snapshot).

Métrica	**Inventário**
Agilidade	
Tempo de construção (ms)	+196 %
Tempo de inicialização (ms)	+32 %
Custo	
CPU total (repouso / carga, %)	+4,38 p.p. / +68 %
RAM total (repouso / carga, MB)	+136 % / +136 %
Imagens da aplicação (MB)	+287 %
Componentes / contêineres	+4
Desempenho	
Vazão (média das rotas, req/s)	+1 %
Latência p50 (média das rotas, ms)	− 12 %
Manutenibilidade	
Linhas de Código	+9 %
Chamadas entre fronteiras	+13 %

Fonte: elaborado pelo autor (2026) a partir da medição automatizada. S0 canônico: Capacidade de Negócio / S0. A variação de CPU em repouso é expressa em pontos percentuais (p.p.); as demais, em variação relativa.

Sistema de Leilões

E-commerce

5.4 Decomposição por Subdomínio (DDD)

Sistema de Inventário

Sob a decomposição por subdomínio, os seis módulos foram classificados pelo valor estratégico que entregam ao negócio, e não pela cadeia de processos da organização. A classificação separou o núcleo do sistema, formado pelas movimentações, dos domínios de suporte, produtos e armazéns, e dos genéricos, que reúnem a identidade e os relatórios. O núcleo é o que carrega o diferencial competitivo e concentra as regras de negócio; o suporte é necessário ao funcionamento, mas não diferencia o sistema; e os genéricos poderiam ser atendidos por componentes prontos. No monólito de referência, a análise estática registrou 69 dependências cruzando fronteiras de módulo, de um total de 389, sendo a maior parte direcionada ao núcleo compartilhado.

A extração seguiu a ordem inversa à da Capacidade de Negócio. Saíram primeiro os dois domínios genéricos, Relatórios e Identidade, por serem os de menor valor estratégico e menor risco. A extração seguiu o padrão *Strangler Fig* mediado por NGINX, com roteamento por prefixo de caminho, e os serviços apontavam para o mesmo PostgreSQL. Na etapa seguinte, saiu o suporte, com produtos e armazéns, de modo que o monólito passou a conter apenas o núcleo. Até essa fase, a persistência permanecia compartilhada

e não havia comunicação em rede entre os serviços.

A etapa final extraiu o núcleo e isolou a persistência, com um banco por serviço e comunicação por chamadas HTTP. Esse arranjo revelou a principal dificuldade da trilha: a regra de estoque exige, ao registrar uma movimentação, a checagem do produto e do armazém, que na decomposição por subdomínio passaram a pertencer ao serviço de suporte. Como o núcleo ficou separado do suporte, registrar uma movimentação passou a depender de validações remotas: o produto e o armazém no Suporte e o usuário na Identidade. Esse é o contraste direto com a Capacidade de Negócio, que mantinha movimentações, produtos e armazéns agrupados e preservava a verificação dentro de um único processo.

A Tabela 5 apresenta as métricas ao longo das quatro fases e a Tabela 6 consolida a variação acumulada da metodologia em relação ao S0 canônico. O tempo de construção subiu de cerca de 47 segundos no baseline para cerca de 123 segundos no estado final, em razão da construção de imagens independentes. O tempo de inicialização variou pouco até a segunda extração e subiu no estado final, quando passaram a existir três bancos. O número de componentes cresceu de dois para oito. As Linhas de Código subiram até a extração do suporte, por duplicação de código transversal, e caíram no estado final, quando o monólito deixou de compor a solução. As chamadas entre fronteiras medidas pela análise estática variaram pouco, pois o acoplamento entre serviços passou a ocorrer pela rede e não é capturado por essa análise. A vazão média das rotas apresentou leve recuperação no estado final e a latência média caiu em relação ao baseline. Vale registrar que a primeira extração desta trilha coincide, em topologia e em números, com a segunda fase da Capacidade de Negócio, o que reforça que o efeito medido até ali decorre da estrutura resultante e não da metodologia empregada.

Tabela 5 – Inventário — métricas por snapshot (Subdomínio (DDD)).

Métrica	**S0**	**S1**	**S2**	**Final**
Agilidade				
Tempo de construção (ms)	46.746	108.628	103.700	122.958
Tempo de inicialização (ms)	9.155	10.056	9.626	12.540
Custo				
CPU total (repouso / carga, %)	2,22 / 241	4,03 / 249,96	2,78 / 251,4	2,52 / 370,73
RAM total (repouso / carga, MB)	86,28 / 188,47	184,79 / 333,04	224,2 / 318,21	268,4 / 538,18
Imagens da aplicação (MB)	151,6	551,2	737,8	772,9
Componentes / contêineres	2 / 2	5 / 5	6 / 6	8 / 8
Desempenho				
Vazão (média das rotas, req/s)	355,3	319,7	304	415,9
Latência p50 (média das rotas, ms)	62,6	70,4	70,1	47,4
Manutenibilidade				
Linhas de Código	8.783	9.796	10.310	10.079
Chamadas entre fronteiras	69 de 389	78 de 414	85 de 429	85 de 431

Fonte: medição automatizada do experimento (média de três execuções). S0 = baseline canônico (Capacidade de Negócio / S0); demais snapshots medidos nesta trilha.

Tabela 6 – Variação acumulada por aplicação — Subdomínio (DDD) (S0 canônico → último snapshot).

Métrica	**Inventário**
Agilidade	
Tempo de construção (ms)	+163 %
Tempo de inicialização (ms)	+37 %
Custo	
CPU total (repouso / carga, %)	+0,3 p.p. / +54 %
RAM total (repouso / carga, MB)	+211 % / +186 %
Imagens da aplicação (MB)	+410 %
Componentes / contêineres	+6
Desempenho	
Vazão (média das rotas, req/s)	+17 %
Latência p50 (média das rotas, ms)	− 24 %
Manutenibilidade	
Linhas de Código	+15 %
Chamadas entre fronteiras	+23 %

Fonte: elaborado pelo autor (2026) a partir da medição automatizada. S0 canônico: Capacidade de Negócio / S0. A variação de CPU em repouso é expressa em pontos percentuais (p.p.); as demais, em variação relativa.

Sistema de Leilões

E-commerce

5.5 Decomposição por Transações

Sistema de Inventário

Na decomposição por transações, o agrupamento parte do que precisa ser gravado de forma atômica. No sistema de inventário, a regra que impede retirar mais do que existe (ADR-0006) e o cálculo de estoque a partir das movimentações (ADR-0008) exigem que a verificação de produto e de armazém e a escrita do movimento ocorram na mesma transação. Por isso, armazéns, produtos e movimentações formaram um único serviço, o Estoque. A Identidade e os Relatórios ficaram fora dessa fronteira: a primeira fornece o usuário referenciado pelo movimento, e os segundos apenas leem os dados. No monólito de referência, a análise estática registrou 69 dependências cruzando fronteiras de módulo, de um total de 389.

A ordem de extração foi o oposto da adotada na Capacidade de Negócio: o grupo atômico saiu primeiro, para que a invariante fosse preservada em um único processo desde o início. A extração seguiu o padrão *Strangler Fig* mediado por NGINX, com as rotas de armazéns, produtos e movimentações desviadas para o novo serviço. Nessa fase, a persistência era compartilhada, e o Estoque passou a validar o usuário no monólito por um endpoint interno. Para manter o e-mail do usuário no histórico de movimentações, foi

adicionada a coluna `user_email` à tabela de movimentos, já que a Identidade ainda não era um serviço separado.

A segunda extração foi a da Identidade. O monólito manteve apenas os Relatórios e uma cópia local dos guardas de autenticação, que validam o token sem consultar outro serviço. O Estoque passou a validar o usuário na Identidade, e a Identidade passou a validar o armazém no Estoque, ambas por chamadas HTTP. A persistência continuou compartilhada, de modo que os Relatórios seguiam lendo os dados de estoque diretamente no banco.

Na etapa final, os Relatórios foram extraídos, o monólito foi desativado e a persistência foi isolada, com um banco para a Identidade e outro para o Estoque. O serviço de Relatórios deixou de ter banco e passou a consultar a API interna do Estoque. A principal dificuldade da trilha foi o acoplamento entre movimento e usuário: como o usuário deixou de estar no mesmo banco do movimento, sua validação passou a ser remota e o e-mail foi desnormalizado no registro da movimentação. É a mesma dificuldade observada na Capacidade de Negócio, mas aqui ela aparece desde a primeira extração.

As Tabelas 7 e 8 apresentam as métricas ao longo das fases e a variação acumulada da metodologia. O tempo de construção subiu de cerca de 47 segundos no baseline canônico para cerca de 112 segundos no estado final, em razão da construção de imagens independentes. O tempo de inicialização subiu de cerca de 9 segundos para cerca de 14 segundos. O número de componentes cresceu de dois para seis. As Linhas de Código subiram até a segunda extração, por duplicação de código transversal, e caíram no estado final, quando o monólito deixou de compor a solução. As chamadas entre fronteiras medidas pela análise estática variaram pouco, pois o acoplamento entre serviços passou a ocorrer pela rede e não é capturado por essa análise. No estado final, a vazão média das rotas ficou cerca de 8% acima do baseline e a latência mediana caiu cerca de 18%.

O estado final coincide com o da Capacidade de Negócio: mesma topologia, mesmas 9.599 Linhas de Código e as mesmas 78 de 416 chamadas entre fronteiras. A diferença está na trajetória. Nesta trilha, o grupo atômico saiu na primeira extração, e o caminho de escrita passou a depender da rede antes das demais metodologias; na Capacidade, a escrita permaneceu local até o estado final. O resultado indica que, neste domínio, o critério de transações converge para a mesma fronteira da capacidade, porque a invariante de estoque domina o agrupamento. A comparação relevante deixa de ser o formato final e passa a ser quando o custo é pago e por quanto tempo a consistência da escrita depende de um único processo.

Tabela 7 – Inventário — métricas por snapshot (Transações).

Métrica	**S0**	**S1**	**S2**	**Final**
Agilidade				
Tempo de construção (ms)	46.746	75.321	99.791	111.724
Tempo de inicialização (ms)	9.155	9.788	12.537	14.028
Custo				
CPU total (repouso / carga, %)	2,22 / 241	1,95 / 259,31	3,13 / 247,83	1,75 / 433,39
RAM total (repouso / carga, MB)	86,28 / 188,47	146,86 / 305,28	185,32 / 327,95	203,03 / 421,79
Imagens da aplicação (MB)	151,6	363,5	551,3	586,6
Componentes / contêineres	2 / 2	4 / 4	5 / 5	6 / 6
Desempenho				
Vazão (média das rotas, req/s)	355,3	329,5	314	384,2
Latência p50 (média das rotas, ms)	62,6	64,9	67,8	51,6
Manutenibilidade				
Linhas de Código	8.783	9.431	9.856	9.599
Chamadas entre fronteiras	69 de 389	77 de 408	81 de 419	78 de 416

Fonte: medição automatizada do experimento (média de três execuções). S0 = baseline canônico (Capacidade de Negócio / S0); demais snapshots medidos nesta trilha.

Tabela 8 – Variação acumulada por aplicação — Transações (S0 canônico → último snapshot).

Métrica	**Inventário**
Agilidade	
Tempo de construção (ms)	+139 %
Tempo de inicialização (ms)	+53 %
Custo	
CPU total (repouso / carga, %)	− 0,47 p.p. / +80 %
RAM total (repouso / carga, MB)	+135 % / +124 %
Imagens da aplicação (MB)	+287 %
Componentes / contêineres	+4
Desempenho	
Vazão (média das rotas, req/s)	+8 %
Latência p50 (média das rotas, ms)	− 18 %
Manutenibilidade	
Linhas de Código	+9 %
Chamadas entre fronteiras	+13 %

Fonte: elaborado pelo autor (2026) a partir da medição automatizada. S0 canônico: Capacidade de Negócio / S0. A variação de CPU em repouso é expressa em pontos percentuais (p.p.); as demais, em variação relativa.

Sistema de Leilões

E-commerce

5.6 Comparação entre as Metodologias

A Tabela 9 reúne a variação acumulada de cada metodologia entre o S0 canônico e o estado final. Como a direção desejável varia entre as métricas (menor custo e latência, maior vazão), a comparação deve ser feita por métrica, e não por uma média geral.

Tabela 9 – Variação acumulada por metodologia (S0 canônico → último snapshot).

Métrica	**Capacidade de Negócio**	**Subdomínio (DDD)**	**Transações**
Agilidade			
Tempo de construção (ms)	+196 %	+163 %	+139 %
Tempo de inicialização (ms)	+32 %	+37 %	+53 %
Custo			
CPU total (repouso / carga, %)	+4,38 p.p. / +68 %	+0,3 p.p. / +54 %	− 0,47 p.p. / +80 %
RAM total (repouso / carga, MB)	+136 % / +136 %	+211 % / +186 %	+135 % / +124 %
Imagens da aplicação (MB)	+287 %	+410 %	+287 %
Componentes / contêineres	+4	+6	+4
Desempenho			
Vazão (média das rotas, req/s)	+1 %	+17 %	+8 %
Latência p50 (média das rotas, ms)	− 12 %	− 24 %	− 18 %
Manutenibilidade			
Linhas de Código	+9 %	+15 %	+9 %
Chamadas entre fronteiras	+13 %	+23 %	+13 %

Fonte: elaborado pelo autor (2026) a partir da medição automatizada. S0 canônico: Capacidade de Negócio / S0. A variação de CPU em repouso é expressa em pontos percentuais (p.p.); as demais, em variação relativa.

5.7 Síntese do Capítulo

6 Conclusão

Este capítulo apresentará as considerações finais, a avaliação dos resultados alcançados pela metodologia proposta e as direções para trabalhos futuros.

6.1 Contribuições

6.2 Limitações

6.3 Trabalhos Futuros

Referências

ABGAZ, Y. et al. Decomposition of monolith applications into microservices architectures: A systematic review. *IEEE Transactions on Software Engineering*, v. 49, p. 4213–4242, 2023. Citado na página 22.

BARZOTTO, T. R. H.; FARIAS, K. Avaliação dos impactos da decomposição de uma aplicação monolítica para microsserviços: Um estudo de caso. *arXiv preprint arXiv:2203.13878*, 2022. Disponível em: <https://arxiv.org/abs/2203.13878>. Citado 2 vezes nas páginas 20 e 22.

EVANS, E. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Boston, MA: Addison-Wesley Professional, 2003. Citado na página 15.

FOWLER, M. *StranglerFigApplication*. 2004. Martinfowler.com. Disponível em: <https://martinfowler.com/bliki/StranglerFigApplication.html>. Acesso em: 18.08.2026. Citado na página 25.

KAZANAVIČIUS, J.; MAŽEIKA, D. Analysis of legacy monolithic software decomposition into microservices. In: *Forum@DB&IS 2020 (Doctoral Consortium)*. [S.l.: s.n.], 2020. (CEUR Workshop Proceedings), p. 25–32. Citado 2 vezes nas páginas 20 e 22.

KITCHENHAM, B. A.; PFLEEGER, S. L.; PICKARD, L. M. Preliminary guidelines for empirical research in software engineering. *IEEE Transactions on Software Engineering*, v. 28, n. 8, p. 721–734, 2002. Citado na página 17.

LEVCOVITZ, A.; TERRA, R.; VALENTE, M. T. Towards a technique for extracting microservices from monolithic enterprise systems. In: *III Brazilian Workshop on Software Visualization, Evolution and Maintenance (VEM)*. [s.n.], 2016. p. 97–104. Acesso em: 10 set. 2026. Disponível em: <https://arxiv.org/abs/1605.03175>. Citado 2 vezes nas páginas 20 e 22.

LEWIS, J.; FOWLER, M. *Microservices: a definition of this new architectural term*. 2014. Martinfowler.com. Disponível em: <https://martinfowler.com/articles/microservices.html>. Acesso em: 18.08.2026. Citado 3 vezes nas páginas 11, 12 e 13.

NEWMAN, S. *Monolith to Microservices: Evolutionary Patterns to Transform Your Monolith*. Sebastopol, CA: O'Reilly Media, 2020. ISBN 978-1-492-07554-7. Citado 4 vezes nas páginas 11, 13, 15 e 17.

NEWMAN, S. *Building Microservices: Designing Fine-Grained Systems*. 2nd. ed. Sebastopol, CA: O'Reilly Media, 2021. Citado 5 vezes nas páginas 11, 12, 13, 14 e 15.

PAGE-JONES, M. Comparing techniques by means of encapsulation and connascence. *Communications of the ACM*, ACM, v. 35, n. 9, p. 147–151, 1992. Citado na página 17.

RICHARDSON, C. *Microservices Patterns: With examples in Java*. Shelter Island, NY: Manning Publications, 2018. ISBN 9781617294549. Citado 6 vezes nas páginas 11, 13, 14, 15, 16 e 17.

SÖYLEMEZ, M.; TEKINERDOGAN, B.; TARHAN, A. K. Challenges and solution directions of microservice architectures: A systematic literature review. *Applied Sciences*, v. 12, n. 11, p. 5507, 2022. Citado na página 13.

VELEPUCHA, V.; FLORES, P. A survey on microservices architecture: Principles, patterns and migration challenges. *IEEE Access*, v. 11, p. 88369–88386, 2023. Citado 2 vezes nas páginas 11 e 12.

WANG, Y.; BORNAIS, S.; RUBIN, J. Microservice decomposition techniques: An independent tool comparison. In: *39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. [S.l.: s.n.], 2024. Citado 2 vezes nas páginas 20 e 22.

WOHLIN, C. et al. *Experimentation in Software Engineering*. [S.l.]: Springer, 2012. Citado na página 17.